

HARDWARE RESEARCH PROPOSAL / 09 SEPTEMBER 2026

Accelerating LLM Serving at the GPU DMA Level

KV-cache transfers for SGLang and similar serving engines

Prepared for Oluwole Jaiyeoba | Concept proposal, version 1.0

IN SIMPLE WORDS
My recommendation is to improve the part of the GPU that moves data, so an important request can get the data it needs sooner.

Develop a DMA engine optimized for scattered KV-cache blocks, with hardware scheduling that can switch from background work to urgent restores at small, safe transfer boundaries. Begin with host-memory-to-GPU and GPU-to-host transfers in a single serving worker.

The objective is faster request resumption and less interference with inference. This is an engineering hypothesis; no hardware speedup or deadline guarantee has been demonstrated by this proposal. Proposed mechanisms throughout this document require implementation and measurement.

The basic picture

Term	Simple meaning
GPU memory	The worker's desk: fast to reach, but limited in space.
Host memory	A larger storage room outside the GPU.
KV cache	The model's saved work from reading earlier parts of a conversation. Reusing it avoids doing that work again.
DMA	Direct Memory Access: the equipment that moves this saved work between the storage room and the desk.

When a conversation's saved work has been moved out of GPU memory, the model may need to bring it back before continuing. KV cache contains attention data, not the model's weights or a complete copy of the application's state.

Four capabilities in one proposed engine

- 1.** Collect scattered blocks efficiently. **2.** Let urgent transfers proceed at chunk boundaries. **3.** Report completion per layer. **4.** Control background bandwidth while preserving progress.

01 / EVIDENCE AND SCOPE

Build on the existing software

The proposal targets a specific part of request latency: moving reusable KV data back into GPU memory. The following documented behavior establishes the baseline. Bracketed source numbers link to the references on page 8.

What already exists

SGLang HiCache. It extends KV caching to host memory and storage. Host layouts can differ from GPU layouts, so transfer organization matters. It offers direct copies and GPU-assisted I/O kernels, and overlaps layer N computation with loading layer N+1 during prefill. [1]

Copy-kernel tradeoff. vLLM's published experiments found that custom GPU copy kernels can move small blocks effectively, while their use of GPU cores can interfere with model computation. DMA can be preferable for serving throughput; results depend on layout and workload. [2]

Priority gap. NVIDIA documents that CUDA stream priorities affect compute kernels, not host-to-device or device-to-host copies. Stream priorities also do not preempt already-running work. A high request priority alone therefore does not establish transfer priority. [3]

Batching baseline. CUDA already provides batched-copy APIs that amortize transfer setup costs. The proposed engine must demonstrate value beyond batching alone. [4]

Where decisions belong

Software: SGLang and a transfer adapter	Hardware: the proposed DMA engine
Find reusable KV blocks; choose restore versus recomputation; allocate destination memory.	Execute copies using the supplied addresses, lengths and valid memory mappings.
Assign urgency, dependencies and transfer budgets; decide what to retain or evict.	Arbitrate ready work, switch at safe boundaries and enforce transfer budgets.
Schedule model computation when data and other dependencies are ready.	Publish completion only after the requested writes are visible to their consumers.

IN SIMPLE WORDS

SGLang decides what matters. The hardware carries out those decisions efficiently. The DMA engine does not need to understand the conversation.

Initial scope: one worker and local host/GPU memory. Peer-GPU transfers are a possible extension. Storage retrieval, network routing, cache-residency policy, compression and speculative execution remain separate workstreams. This proposal does not redesign the HBM-to-shared-memory path inside attention kernels.

02 / HARDWARE CAPABILITIES

Collect efficiently. Switch promptly.

1. Efficient scatter/gather

IN SIMPLE WORDS

A conversation's saved work may be stored in many different places.

Give the DMA engine a list of those places and let it collect the pieces efficiently. This could reduce the organizing work required from the CPU and reduce the need to use GPU computing cores to help move data.

Technical design. Accept batched source/destination descriptors or indexed block lists. Dedicated address-generation logic walks those lists, coalesces compatible adjacent ranges and maintains enough outstanding transactions to use the link efficiently. Copy directly into allocated KV slots, with explicit strides or layout mappings where needed.

Expected benefit. Lower per-block submission overhead and less use of GPU compute cores for copying. Scatter/gather does not eliminate address translation, fragmentation or the physical cost of moving bytes. Keep mappings valid and retain buffers until their transfers complete.

What to prove. An advantage over tuned batched DMA and copy kernels across realistic block sizes and layouts. Scatter/gather itself is not claimed as a new invention.

2. Priority with chunk-level interruption

IN SIMPLE WORDS

Suppose a large background transfer is underway when an important request needs its saved work.

The improved DMA would finish the small piece it is currently moving, pause the background transfer, handle the urgent transfer, and then continue the background job.

Technical design. Maintain separate urgency queues and retain progress within each descriptor. Divide large logical transfers into bounded chunks. At a safe boundary, choose a ready urgent chunk before further background chunks. Bound background work already issued to downstream resources; queued data cannot simply be recalled.

Expected benefit. Shorter urgent-transfer waiting time when traffic shares a constrained engine, link or memory resource. Preserve ordering for dependent or overlapping writes. Prioritize any transfer needed to free the urgent request's destination space.

What to prove. Measure switching delay under contention while sweeping chunk size and outstanding-work limits. Small chunks improve responsiveness but can lower throughput. The design goal is a measured latency bound under stated conditions, not instantaneous interruption.

03 / HARDWARE CAPABILITIES

Start earlier. Control contention.

3. Completion signals per layer

IN SIMPLE WORDS

A model processes information through a series of layers.

Instead of waiting for the entire saved context to arrive, it could start a layer once all the saved data that layer needs is ready. Meanwhile, DMA brings in the data for later layers.

Technical design. Associate descriptors with a completion group for a request and layer. Track outstanding bytes or blocks and signal the group only after every required write is complete and visible. Integrate those signals with the serving engine's execution dependencies, so later-layer transfers can overlap earlier-layer computation.

Expected benefit. Reduce the portion of restore time exposed to the user. A layer must still wait for all data required by its attention operation. Scheduler admission, activation dependencies and execution-graph constraints can also delay its start.

Existing baseline. SGLang already supports layer-wise prefill overlap. [1] The proposed contribution is lower-overhead movement and usable completion signaling. It does not assume every decode path or model can adopt this schedule unchanged.

4. Bandwidth control and continued progress

IN SIMPLE WORDS

Moving data and running the model can compete for memory bandwidth.

The DMA controller would limit background traffic when necessary, while still allowing it to make progress.

Technical design. Use programmable byte budgets, outstanding-request limits and weighted arbitration across traffic classes. A minimum background share or aging policy prevents starvation. Software sets policy and can adjust budgets from telemetry. Track host-link pressure separately from GPU-memory pressure.

Expected benefit. Reduce stalls caused by background copies while keeping transfers useful. Hardware rate limiting controls offered DMA load; predictable service for compute may also require cooperation from shared interconnect and memory arbiters.

What to prove. Measure token-generation gaps, urgent restore latency, background completion time and total throughput together. Throttling can delay KV eviction and memory reclamation, so dependency-critical writebacks must remain eligible for prompt service.

These four capabilities are proposed as one coordinated data-movement engine. Their benefits may overlap; individual improvements should not be added together as if they were independent speedups.

04 / INTEGRATION AND EXAMPLE

An agent returns from a tool call

Illustrative scenario. Agent A is waiting for a tool result. Assume its reusable KV context has been placed in host memory and destination GPU space can be made available. Other requests are moving data. When the tool finishes, A is marked urgent by software. Tool-wait offload is a scenario assumption, not a claim that SGLang automatically applies this policy.

Step	What happens
1. Identify	SGLang finds A's reusable prefix, identifies missing blocks and allocates GPU destinations.
2. Describe	A transfer adapter submits addresses, sizes, priority and layer completion groups.
3. Switch	DMA finishes the current safe background chunk, then gives A's ready transfers service.
4. Restore	DMA brings in the earliest required layer data and signals only when that group is complete.
5. Resume	SGLang starts eligible computation while later layers continue loading, where its execution path supports overlap.
6. Continue	Background transfers proceed with remaining capacity and guaranteed progress under the configured policy.

IN SIMPLE WORDS

When Agent A needs to continue, the moving equipment brings back A's saved work promptly. The model can resume sooner if those transfers were what was holding it up.

Proposed transfer interface

These are conceptual descriptor fields, not existing SGLang API parameters. An implementation would require a software adapter and a driver/runtime interface to the new hardware.

Field group	Purpose
Addresses and sizes	Source/destination ranges or block-list handles; layout or stride information; transfer direction.
Identity and urgency	Request ID and priority class; an optional deadline supplied by software.
Dependencies	Layer/completion-group ID; prerequisites and valid buffer lifetime.
Service controls	Maximum chunk size, traffic class and applicable bandwidth budget.
Completion and telemetry	Status, bytes completed, queue delay and completion timestamps.

Software retains responsibility for shared-prefix ownership and prevents in-flight buffers from being freed or reused. Multi-GPU deployment would add cross-rank readiness coordination; a local DMA completion alone is insufficient.

05 / BENEFITS AND LIMITS

Where the improvement can matter

Expected outcomes below are hypotheses to test. No numerical speedup is asserted. A faster transfer engine improves user experience only when it reduces time on the request's critical path.

Potential outcome	Mechanism	How to measure it
Faster urgent resumption	Urgent KV restores encounter less avoidable blocking.	Tool-result arrival to first resumed output token; p50, p95 and p99.
Lower transfer overhead	Hardware processes block lists with fewer small software submissions.	CPU submission time, GPU copy-kernel time and useful bytes per second.
Fewer computation stalls	Layer readiness exposes usable data earlier; background traffic is regulated.	Time waiting for KV and gaps between generated tokens.
Better useful throughput	Copying uses fewer compute resources and less disruptive traffic.	Completed requests or tokens per second, including those meeting the latency target.

Conditions that make the idea promising

The strongest initial workload has reusable KV in host memory, repeated restores, scattered blocks and competing transfers. Long-context or multi-turn agent traffic can provide these conditions, but the actual traces must confirm them. A tool call is useful as an arrival pattern; DMA cannot shorten the tool's own execution time.

Practical limits

- Everything is already resident.** If required KV remains in GPU memory, this host/GPU DMA redesign offers little direct benefit.
- The physical link is full.** When equally urgent work already saturates the link, priority cannot create bandwidth. Address handling may help only if software or fragmentation was leaving capacity unused.
- Another stage dominates.** CPU admission, scheduling, storage retrieval, network delay, computation or attention's GPU-memory reads may determine latency instead.
- Transfers share resources.** Pausing one copy may not help if the relevant bottleneck is elsewhere. Even dedicated DMA consumes host and GPU memory bandwidth.
- There is no free destination.** The urgent restore may first require memory reclamation. Offload dependencies must receive suitable priority too.

IN SIMPLE WORDS

The benefit is conditional: this helps when requests spend meaningful time waiting for data transfers. It offers much less benefit when everything they need is already in GPU memory.

06 / EVALUATION PLAN

Prove value beyond good software

IN SIMPLE WORDS

Before building hardware, compare the idea against the best existing software approach. The question is: does it make important requests resume faster while maintaining good performance for everyone else?

1. Establish the measured bottleneck

Pin software versions, model, KV layout, hardware topology and allocation policy. Begin with one worker. Instrument transfer submission, queueing, layer readiness, execution and token output. Measure the waiting that remains after existing overlap.

2. Build strong software baselines

Compare the best supported HiCache direct and kernel paths, tuned batching and layouts, and a software scheduler that chunks copies and prioritizes submissions. Include an all-KV-resident control when feasible. Software chunking can approximate scheduling policy, but cannot demonstrate the proposed hardware's descriptor cost or interruption latency.

3. Evaluate the proposed mechanisms separately

Use a calibrated simulator or prototype to vary descriptor processing, chunk size, outstanding transfers, layer completion and bandwidth budgets. Test capabilities separately and together. Preserve measured link and memory limits; include scheduling cost. Label measured results and modeled projections separately.

4. Replay serving workloads and inspect tradeoffs

Sweep context length, physical block size, request concurrency, host-cache hit rate, urgent-arrival bursts and background read/write traffic. Include both isolated transfer tests and concurrent inference. Verify exact copy results and correct buffer lifetimes under switching, cancellation and resource pressure.

Report the tradeoffs together

Report p50/p95/p99 resume latency, KV waiting, token gaps, total throughput, deadline success, CPU/GPU copy overhead, useful bandwidth, switching delay and background completion time. Compare equal offered load and memory budgets; include repeated runs and variability.

Decision criterion. Advance toward hardware only if it improves the chosen latency objective beyond the tuned software baseline, with an acceptable measured throughput/fairness tradeoff and credible area, power and integration cost. Choose numeric acceptance thresholds from the target workload before evaluation.

07 / SOURCES AND INTERPRETATION

Evidence behind the proposal

Primary sources reviewed on 09 September 2026. These describe existing software behavior; they do not validate the proposed hardware design. Links are clickable. No benchmark from these sources is presented as a predicted speedup for this proposal.

[1] SGLang documentation

HiCache System Design and Optimization

[Open official source](#)

Supports the discussion of host/GPU KV layouts, I/O backends and layer-wise transfer/computation overlap.

[2] vLLM project blog

Inside vLLM's New KV Offloading Connector: Smarter Memory Transfer for Maximizing Inference Throughput

[Open official source](#)

Published 08 January 2026. Supports the comparison of DMA and GPU copy kernels, including layout sensitivity and compute interference.

[3] NVIDIA CUDA Runtime API

Stream Management: `cudaStreamCreateWithPriority`

[Open official source](#)

Supports the limitation that stream priority does not affect host/device memory copies and does not preempt already-running work.

[4] NVIDIA CUDA Programming Guide

Advanced CUDA APIs and Features

[Open official source](#)

Supports inclusion of batched memory-copy APIs as an existing software baseline.

How to interpret the recommendations

The four hardware capabilities, interface fields, workload example and evaluation plan are proposed design choices. They are engineering inferences motivated by the sources and the preceding discussion. They are not statements that current NVIDIA hardware exposes this interface, that current SGLang implements these policies, or that these mechanisms are novel individually.

Terms used in the evaluation

p50 / p95 / p99: the delay at or below which 50%, 95% or 99% of observations fall. **Throughput:** work completed per second. **Bandwidth:** data moved per second. **Critical path:** the dependencies that determine when a request can finish. **Starvation:** lower-priority work repeatedly being denied service.